

Experiment No. 9

Write the Python program to implement the A* Algorithm

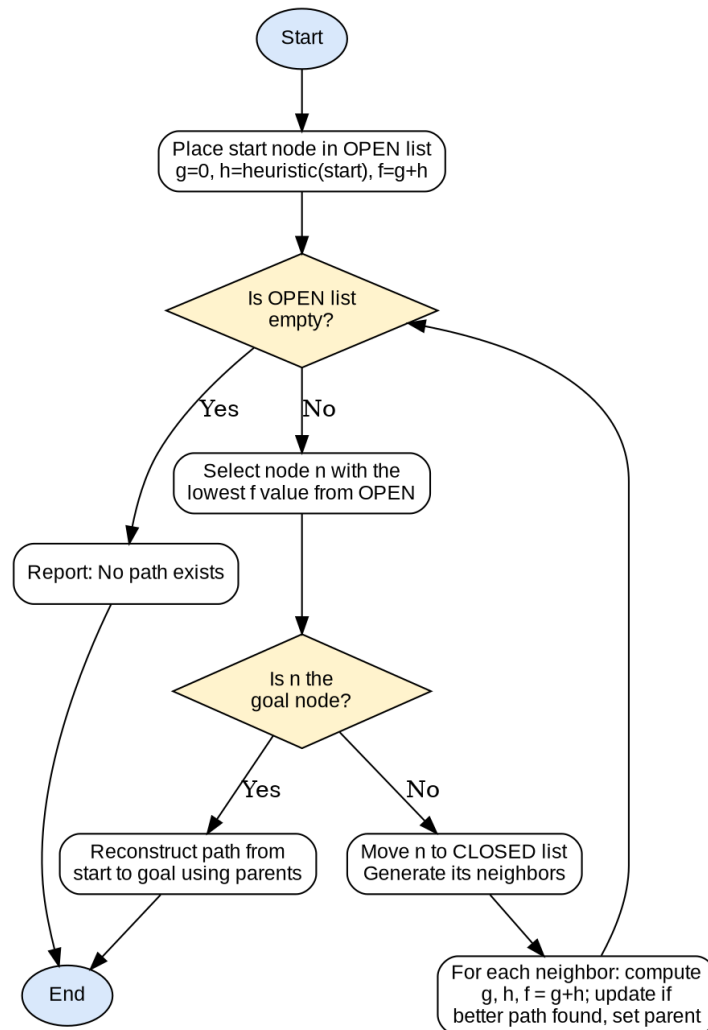
Aim

To write a Python program to implement the A* search algorithm to find the least-cost path between a start node and a goal node in a weighted graph, using a heuristic function.

Algorithm

1. Start.
2. Represent the search space as a graph with edge costs, and define a heuristic function $h(n)$ that estimates the cost from each node to the goal.
3. Initialize the OPEN list with the start node; set $g(\text{start}) = 0$ and $f(\text{start}) = g(\text{start}) + h(\text{start})$.
4. Initialize an empty CLOSED list.
5. Repeat while the OPEN list is not empty:
 6. (a) Select the node n with the lowest f value from the OPEN list.
 7. (b) If n is the goal node, reconstruct the path using parent pointers and stop.
 8. (c) Otherwise, move n to the CLOSED list and generate its neighboring nodes.
 9. (d) For each neighbor, compute g , h and $f = g + h$; if this path to the neighbor is better than any found earlier, update its g , f and parent, and place it in the OPEN list.
10. If the OPEN list becomes empty before the goal is reached, report that no path exists.
11. Display the path found and its total cost.
12. Stop.

Flowchart



Program

```
import heapq
```

```
def astar(graph, heuristics, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristics[start], start))
    g_score = {start: 0}
    parent = {start: None}
    closed = set()

    while open_list:
        f, current = heapq.heappop(open_list)

        if current == goal:
```

```

        path = []
        while current is not None:
            path.append(current)
            current = parent[current]
        return path[::-1], g_score[goal]

    closed.add(current)

    for neighbor, cost in graph[current]:
        if neighbor in closed:
            continue
        tentative_g = g_score[current] + cost
        if neighbor not in g_score or tentative_g <
g_score[neighbor]:
            g_score[neighbor] = tentative_g
            f_score = tentative_g + heuristics[neighbor]
            parent[neighbor] = current
            heapq.heappush(open_list, (f_score,
neighbor))

    return None, float('inf')

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('D', 2), ('E', 5)],
    'C': [('A', 4), ('F', 3)],
    'D': [('B', 2)],
    'E': [('B', 5), ('F', 1)],
    'F': [('C', 3), ('E', 1)]
}

heuristics = {'A': 6, 'B': 4, 'C': 3, 'D': 5, 'E': 2, 'F':
0}

path, cost = astar(graph, heuristics, 'A', 'F')
print("Path found:", " -> ".join(path))
print("Total cost:", cost)

```

Result

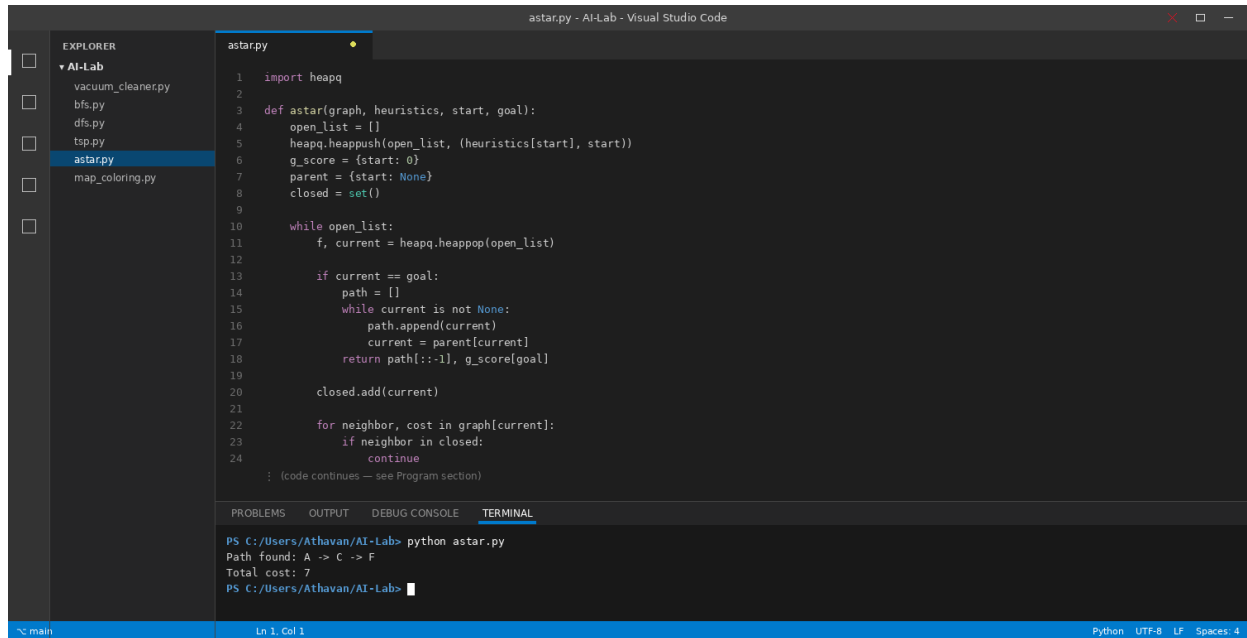
The program was executed successfully and the output was verified as correct.

Path found: A -> C -> F

Total cost: 7

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:



```
1 import heapq
2
3 def astar(graph, heuristics, start, goal):
4     open_list = []
5     heapq.heappush(open_list, (heuristics[start], start))
6     g_score = {start: 0}
7     parent = {start: None}
8     closed = set()
9
10    while open_list:
11        f, current = heapq.heappop(open_list)
12
13        if current == goal:
14            path = []
15            while current is not None:
16                path.append(current)
17                current = parent[current]
18            return path[::-1], g_score[goal]
19
20        closed.add(current)
21
22        for neighbor, cost in graph[current]:
23            if neighbor in closed:
24                continue
25
26    : (code continues — see Program section)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

```
PS C:/Users/Athavan/AI-Lab> python astar.py
Path found: A -> C -> F
Total cost: 7
PS C:/Users/Athavan/AI-Lab>
```

Ln 1, Col 1 Python UTF-8 LF Spaces: 4